The EAOS methodology defines a structured lifecycle for
Designing, deploying, and governing enterprise-grade AI systems

Project: Standard Framework Project

0.1.4 AI Agent Boilerplate Guide

Phase: 0. SaaS Foundation

[산출물 ID: 0.1.4 AI Agent Boilerplate Guide.docx]

① Phase 3 Capability 구축 전 반드시 완비해야 하는 공통 기반 파일의 구조, 구현 순서, 수정 절차를 정의한다..
② ⚠ 선행 조건: 📘 Framework, 📗 Implementation Guide(doc 1~3) 완료 후 이 문서를 따른다.
③ ⚠ Phase 3 착수 전 체크포인트: Section 6의 체크리스트 전체 통과 후 착수 가능.

문서 번호	0.1.4 AI Agent Boilerplate Guide.docx
버전	v1.0
작성일	2026
작성자	KY Song
분류	CONFIDENTIAL
적용 대상	Standard Framework 프로젝트 전체

Table of Contents

1. 문서 개요 3
1.1 기존 doc 1~3 과의 관계 3
1.2 기술 스택 3
1.3 전체 작업 흐름 4
2. Phase 1 — 인프라 / 루트 설정 5
2.1 디렉토리 구조 5
2.2 docker-compose.yml 6
2.3 .env.example 7
2.4 Makefile 8
3. Phase 2 — Backend 공통 기반 9
3.1 Backend 디렉토리 전체 구조 9
3.2 추가 순서 및 파일 상세 10
4. Phase 3 — AI Agent Governance 기반 16
4.1 Governance 파일 목록 (8 개) 16
4.2 파일 의존 관계 17
5. Phase 4 — Frontend 공통 기반 27
5.1 Frontend 디렉토리 구조 27
5.2 주요 파일 28
6. Phase 5 — 통합 연결 및 검증 30
6.1 인프라 실행 30
6.2 Backend 검증 30
6.3 Frontend 검증 30
6.4 Backend 정적 분석 30
7. 전체 작업 체크리스트 31
Phase 1 — 인프라 31
Phase 2 — Backend 공통 기반 31
Phase 3 — AI Agent Governance 기반 31
Phase 4 — Frontend 공통 기반 32
Phase 5 — 통합 검증 32
8. Boilerplate 수정 & 운영 절차 33
8.1 수정 전 확인 사항 33
8.2 파일별 수정 절차 33
8.3 자주 쓰는 명령어 모음 34
8.4 phase3_context.json 업데이트 기준 34
8.5 수정 완료 후 체크리스트 35
9. 참고사항 및 주의점 35
9.1 의존성 순서를 반드시 지켜야 하는 이유 35
9.2 PK 는 반드시 UUID 35
9.3 환경변수 관리 35
9.4 SQLAlchemy async 사용 시 35
9.5 절대 금지 사항 요약 36
9.6 Cursor AI 참조 지침 36
문서 연결 37

1. 문서 개요

1.1 기존 doc 1~3 과의 관계

문서	역할
doc 1. Framework	기술 스택 결정 (Why & What)
doc 2. Implementation Guide	공통 SaaS 기반 구현 순서 (How to Build)
doc 3. Quick Reference	네이밍 · 에러 코드 · API 패턴 (Lookup)
doc 4. 이 문서	**공통 기반 전체 + AI Agent Governance 전용 기반**

doc 2 가 일반 SaaS 공통 기반을 다룬다면, 이 문서는 **인프라 설정부터 AI Agent Governance 구조까지 전체 Boilerplate 를 통합** 한다.

⚠□ doc2 Phase5 중돌 주의

doc2(Implementation Guide)의 **Phase5 — AI Agent 도메인** 과 이 문서의 **Phase3 — AI Agent Governance 기반**은 둘 다 "AI Agent 코드"를 다루지만 **구조가 다르다.**

항목	doc2 Phase5 (일반 SaaS Agent)	이 문서 Phase3 (Governance Agent) ← 우선 적용
Agent 모델	AgentType(general · analyst · coder)	AuthorityLevel(L0~L3)
실행 구조	guardrail.validate_input() 단순 호출	BaseAgent → Orch → HITL(P1/P2/P3)
Authority	없음	L0~L3 명시 · 강제
승인 게이트	없음	ApprovalGate 필수

> 규칙: AI Agent 코드 생성 시 doc2 Phase5 가 아닌 이 문서(doc4) Phase3 을 기준으로 한다.
> doc2 Phase5 는 일반 SaaS 용 참고 자료로만 활용하고, Governance Agent 구현에는 사용하지 않는다.
> Cursor AI 는 이 규칙을 08_uiux.mdc 및 04_saas_stack.mdc 와 함께 적용한다.

1.2 기술 스택

구분	기술
Backend	Python · FastAPI · SQLAlchemy 2.0 (async) · Pydantic
Frontend	TypeScript · React · Next.js (App Router)
DB	PostgreSQL 16
Cache	Redis 7

구분	기술
인프라	Docker Compose (로컬 개발)

1.3 전체 작업 흐름

```
Phase 1. 인프라 / 루트 설정
      ↓
Phase 2. Backend 공통 기반 (FastAPI Core)
      ↓
Phase 3. AI Agent Governance 기반 (base_agent · llm_provider · orchestrator 등)
      ↓
Phase 4. Frontend 공통 기반 (Next.js)
      ↓
Phase 5. 통합 연결 및 검증
      ↓
Phase 3 WBS 착수 가능
```

> 각 Phase는 이전 Phase 완료 후 시작한다. Phase 내부는 의존성 순서(낮은 것 → 높은 것)를 반드시 따른다.

2. Phase 1 — 인프라 / 루트 설정

모든 서비스(Backend, Frontend, DB, Cache)가 이 설정을 기반으로 실행된다. **나머지 모든 Phase 보다 먼저 완료해야 한다.**

2.1 디렉토리 구조

```
project-root/
├── docker-compose.yml              서비스 컨테이너 정의
├── docker-compose.override.yml     로컬 개발 전용 오버라이드
├── .env.example                    환경변수 템플릿 (팀 공유)
├── .gitignore                      Git 추적 제외 파일 목록
└── Makefile                        자주 쓰는 명령어 단축키 모음
```

2.2 docker-compose.yml

PostgreSQL, Redis, Backend(FastAPI), Frontend(Next.js) 4 개 컨테이너를 정의한다.

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - ´5432:5432´

  redis:
    image: redis:7-alpine
    ports:
      - ´6379:6379´

  backend:
    build: ./backend
    env_file: .env
    depends_on: [db, redis]
    ports:
      - ´8000:8000´

  frontend:
    build: ./frontend
    env_file: .env
    depends_on: [backend]
    ports:
      - ´3000:3000´

volumes:
  postgres_data:
```

2.3 .env.example

실제 .env 는 Git 에 올리지 않으므로, 팀원이 필요한 환경변수를 파악할 수 있도록 반드시 작성한다.

```
# Database
POSTGRES_DB=myapp
POSTGRES_USER=postgres
POSTGRES_PASSWORD=changeme
DATABASE_URL=postgresql+asyncpg://postgres:changeme@db:5432/myapp

# Redis
REDIS_URL=redis://redis:6379/0

# Application
SECRET_KEY=your-secret-key-32-chars-minimum
DEBUG=true
CORS_ORIGINS=http://localhost:3000

# LLM
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8000
```

2.4 Makefile

```
up:
	docker compose -f docker-compose.dev.yml up -d

down:
	docker compose -f docker-compose.dev.yml down

logs:
	docker compose -f docker-compose.dev.yml logs -f

restart-be:
	docker compose -f docker-compose.dev.yml restart backend

migrate:
	docker compose -f docker-compose.dev.yml exec backend \
	  alembic upgrade head

shell-be:
	docker compose -f docker-compose.dev.yml exec backend bash

shell-db:
	docker compose -f docker-compose.dev.yml exec db \
	  psql -U postgres -d myapp

test-be:
	docker compose -f docker-compose.dev.yml exec backend pytest app/tests/

lint-be:
	docker compose -f docker-compose.dev.yml exec backend bash -c "ruff check app/ && mypy app/"

lint-fe:
	docker compose -f docker-compose.dev.yml exec frontend bash -c "npx tsc --noEmit && npm run lint"
```

3. Phase 2 — Backend 공통 기반

Backend 파일은 반드시 **의존성 순서** 에 따라 추가해야 한다. 하위 레이어(설정 · DB) 없이 상위 레이어(서비스 · 라우터)를 만들면 ImportError 가 발생한다.

3.1 Backend 디렉토리 전체 구조

```
backend/
├── pyproject.toml
├── .env.example
└── app/
    ├── main.py                 FastAPI 앱 진입점
    ├── core/
    │   ├── config.py           환경변수 관리 (Pydantic BaseSettings)
    │   ├── logging.py          공통 로거 설정
    │   ├── security.py         JWT 인증 · RBAC · api_success/api_error
    │   ├── cache.py            Redis 연결 및 래퍼
    │   ├── exceptions.py       커스텀 예외 클래스
    │   └── audit.py            감사 로그 자동화
    ├── db/
    │   ├── base.py             SQLAlchemy DeclarativeBase
    │   ├── session.py          async engine · AsyncSession · get_db
    │   └── migrations/         Alembic 마이그레이션
    ├── models/
    │   └── base.py             공통 컬럼 (UUID PK · tenant_id · timestamps)
    ├── schemas/
    │   └── base.py             ApiResponse · PaginatedResponse
    ├── utils/
    │   ├── pagination.py
    │   └── datetime.py
    └── api/
        └── deps.py             공통 FastAPI 의존성
```

3.2 추가 순서 및 파일 상세

Step 1 — core/config.py

다른 모든 파일에서 import 하므로 **가장 먼저** 작성한다.

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    database_url: str
    redis_url: str = "redis://localhost:6379/0"
    secret_key: str
    debug: bool = False
    cors_origins: list[str] = ["http://localhost:3000"]
    anthropic_api_key: str = ""
    openai_api_key: str = ""
    access_token_expire_minutes: int = 15
    refresh_token_expire_days: int = 30

    class Config:
        env_file = ".env"

@lru_cache()
def get_settings() -> Settings:
    return Settings()

settings = get_settings()
```

Step 2 — core/logging.py

```
import logging
import sys

def setup_logging():
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=[logging.StreamHandler(sys.stdout)]
    )

def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)
```

Step 3 — db/base.py + db/session.py

```
# db/base.py
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

```
# db/session.py
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.orm import sessionmaker
```

```
from app.core.config import settings

engine = create_async_engine(settings.database_url, echo=settings.debug)

AsyncSessionLocal = sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False
)

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

Step 4 — Alembic 초기화

```
# backend 컨테이너 안에서 실행
alembic init app/db/migrations

# alembic.ini — sqlalchemy.url 을 env.py 에서 동적으로 읽도록 변경
# sqlalchemy.url = (비워둠)
```

Step 5 — models/base.py

> ⚠□ PK 는 UUID — Integer autoincrement 사용 금지 (saas_foundation.json 기준)

```
from sqlalchemy import Boolean, DateTime, func
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from uuid import UUID, uuid4
from app.db.base import Base

class TenantBaseModel(Base):
    """모든 Tenant 격리 테이블의 부모. tenant_id 자동 포함."""
    __abstract__ = True

    id:          Mapped[UUID] = mapped_column(
        PGUUID(as_uuid=True), primary_key=True, default=uuid4
    )
    tenant_id:   Mapped[UUID] = mapped_column(
        PGUUID(as_uuid=True), nullable=False, index=True
    )
    created_at: Mapped[DateTime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
    updated_at: Mapped[DateTime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )
    created_by: Mapped[UUID | None] = mapped_column(PGUUID(as_uuid=True), nullable=True)
    is_active:   Mapped[bool] = mapped_column(Boolean, default=True)
    is_deleted: Mapped[bool] = mapped_column(Boolean, default=False)

class SoftDeleteMixin:
    is_active:   Mapped[bool]
    is_deleted: Mapped[bool]
```

```
    def soft_delete(self) -> None:
        self.is_active  = False
        self.is_deleted = True
```

Step 6 — schemas/base.py

Backend 의 응답 형식은 Frontend types/api.ts 와 반드시 동일한 구조로 유지한다.

```
from pydantic import BaseModel
from typing import Generic, TypeVar, Optional

T = TypeVar("T")

class ApiResponse(BaseModel, Generic[T]):
    success: bool = True
    data: Optional[T] = None
    message: str = ""

class PaginatedResponse(BaseModel, Generic[T]):
    items: list[T]
    total: int
    page: int
    size: int
    total_pages: int
```

Step 7 — core/cache.py

```
import redis.asyncio as redis
import json
from app.core.config import settings

redis_client: redis.Redis = None

async def get_redis() -> redis.Redis:
    global redis_client
    if redis_client is None:
        redis_client = redis.from_url(settings.redis_url)
    return redis_client

async def cache_get(key: str):
    r = await get_redis()
    value = await r.get(key)
    return json.loads(value) if value else None

async def cache_set(key: str, value, ttl: int = 300):
    r = await get_redis()
    await r.setex(key, ttl, json.dumps(value, default=str))

async def cache_delete(key: str):
    r = await get_redis()
    await r.delete(key)
```

Step 8 — core/exceptions.py

```
from fastapi import HTTPException, status

class NotFoundException(HTTPException):
    def __init__(self, detail: str = "Resource not found"):
        super().__init__(status_code=status.HTTP_404_NOT_FOUND, detail=detail)

class UnauthorizedException(HTTPException):
    def __init__(self, detail: str = "Unauthorized"):
        super().__init__(status_code=status.HTTP_401_UNAUTHORIZED, detail=detail)

class ForbiddenException(HTTPException):
    def __init__(self, detail: str = "Forbidden"):
        super().__init__(status_code=status.HTTP_403_FORBIDDEN, detail=detail)

class ConflictException(HTTPException):
    def __init__(self, detail: str = "Conflict"):
        super().__init__(status_code=status.HTTP_409_CONFLICT, detail=detail)

class ScopeViolationError(Exception):
    """allowed_tools 밖 Tool 호출 시도."""

class ApprovalRequiredError(Exception):
    """L1 Agent 미승인 실행 시도."""
```

Step 9 — core/security.py

```
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timezone, timedelta
from uuid import UUID
import secrets
from app.core.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto", bcrypt__rounds=12)

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(user_id: UUID, org_id: UUID, role: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.access_token_expire_minutes
    )
    return jwt.encode(
        {"sub": str(user_id), "org_id": str(org_id), "role": role,
         "jti": secrets.token_urlsafe(16), "exp": expire, "type": "access"},
        settings.secret_key, algorithm="HS256"
    )
```

```
def require_role(*roles: str):
    """FastAPI 의존성 — 허용 Role 검증."""
    from fastapi import Depends, HTTPException
    def _check(token_data = Depends(verify_token)):
        if token_data.get("role") not in roles:
            raise HTTPException(status_code=403, detail="권한이 없습니다")
        return token_data
    return _check

def api_success(data=None, message: str = "") -> dict:
    return {"success": True, "data": data, "message": message}

def api_error(code: str, message: str, status_code: int = 400) -> dict:
    return {"success": False, "error": {"code": code, "message": message}}

def verify_token(token: str) -> dict:
    try:
        return jwt.decode(token, settings.secret_key, algorithms=["HS256"])
    except JWTError:
        raise UnauthorizedException("토큰이 유효하지 않습니다")
```

Step 10 — core/audit.py

```
from uuid import UUID
from sqlalchemy.ext.asyncio import AsyncSession

async def create_audit_log(
    db: AsyncSession,
    tenant_id: UUID,
    user_id: UUID,
    action: str,
    resource_type: str,
    resource_id: UUID | None = None,
    detail: dict | None = None,
) -> None:
    """모든 상태 변경 시 반드시 호출. 물리 삭제 금지."""
    from app.db.models import AuditLog
    log = AuditLog(
        tenant_id=tenant_id, user_id=user_id,
        action=action, resource_type=resource_type,
        resource_id=resource_id, detail=detail or {}
    )
    db.add(log)
    await db.flush()
```

Step 11 — api/deps.py

```
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db

async def get_database(db: AsyncSession = Depends(get_db)):
    yield db
```

```
class PaginationParams:
    def __init__(self, page: int = 1, size: int = 20):
        self.page   = max(1, page)
        self.size   = min(100, max(1, size))
        self.offset = (self.page - 1) * self.size
```

Step 12 — main.py

모든 파일이 준비된 후 **마지막** 에 작성한다.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.core.logging import setup_logging
from app.db.session import engine

@asynccontextmanager
async def lifespan(app: FastAPI):
    setup_logging()
    yield
    await engine.dispose()

app = FastAPI(lifespan=lifespan, title="AI Agent Framework", debug=settings.debug)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 라우터 등록 (도메인 추가 시 여기에 include_router 추가)
# from app.api.v1 import router as v1_router
# app.include_router(v1_router, prefix="/api/v1")

@app.get("/health")
async def health():
    return {"status": "ok"}
```

4. Phase 3 — AI Agent Governance 기반

> 💡 이 Phase 가 이 문서의 핵심. phase3_context.json · workflow_context.json · hitl_context.json 을 반드시 @참조.

4.1 Governance 파일 목록 (8 개)

```
backend/src/
├── core/
│   ├── base_agent.py          ① 모든 Agent 의 부모 클래스
│   └── llm_provider.py        ② LLM 호출 추상화 인터페이스
├── agents/
│   └── orchestrator_agent.py  ③ Orch Agent — 단일 진입점
└── db/
    └── governance_models.py   ④ Governance DB 스키마 (approval / review / override)

frontend/src/
├── types/
│   └── governance.ts          ⑤ AgentMessage · ApprovalState 타입 (단일 출처)
├── hooks/
│   └── useAgent.ts            ⑥ Agent API 호출 단일 훅
└── components/
    └── ApprovalGate.tsx       ⑦ Human 승인 UI (P1/P2/P3)
```

> base_model.py(Phase 2 Step 5)는 Phase 2 에서 이미 구현. Governance 파일들은 이를 상속한다.

4.2 파일 의존 관계

```
base_model.py (Phase 2)
    └── base_agent.py
        ├── llm_provider.py
        └── orchestrator_agent.py

governance.ts  (Frontend — 최상위 타입)
    └── useAgent.ts
        └── ApprovalGate.tsx
```

> 생성 순서: base_agent → llm_provider → orchestrator_agent → governance_models → governance.ts → useAgent.ts → ApprovalGate.tsx

---

① base_agent.py

경로: backend/src/core/base_agent.py

```python
from __future__ import annotations
from abc import ABC, abstractmethod
from enum import Enum
from uuid import UUID, uuid4
from datetime import datetime, timezone
from typing import Any
from dataclasses import dataclass, field

class AuthorityLevel(str, Enum):
    L0 = "L0"   # 자동 실행 가능 (탐지·알림)
    L1 = "L1"   # Human 승인 필수
    L2 = "L2"   # 설계 승인 (Architect)
    L3 = "L3"   # Governance Board 승인

class RiskLevel(str, Enum):
    LOW      = "LOW"
    MEDIUM   = "MEDIUM"
    HIGH     = "HIGH"
    CRITICAL = "CRITICAL"

class MessageType(str, Enum):
    REQUEST      = "REQUEST"
    RESPONSE     = "RESPONSE"
    ESCALATION   = "ESCALATION"
    NOTIFICATION = "NOTIFICATION"

@dataclass
class AgentContext:
    trace_id:        UUID
    agent_code:      str
    user_id:         UUID
    context_version: str
    risk_level:      RiskLevel = RiskLevel.LOW
    payload:         dict      = field(default_factory=dict)
```

```
    parent_trace_id: UUID | None = None

@dataclass
class AgentResult:
    success:          bool
    data:             Any
    trace_id:         UUID
    risk_level:       RiskLevel = RiskLevel.LOW
    requires_approval: bool     = False
    message:          str       = ""

@dataclass
class AgentMessage:
    """Agent 간 통신 표준 메시지 — 12 개 필드 모두 필수."""
    message_id:       UUID      = field(default_factory=uuid4)
    sender_agent:     str       = ""
    receiver_agent:   str       = ""        # agent_code 또는 'human'
    message_type:     MessageType = MessageType.REQUEST
    payload:          dict      = field(default_factory=dict)
    context_version:  str       = ""
    risk_level:       RiskLevel = RiskLevel.LOW
    requires_approval: bool     = False
    created_at:       datetime  = field(
        default_factory=lambda: datetime.now(timezone.utc)
    )
    trace_id:         UUID      = field(default_factory=uuid4)
    parent_message_id: UUID | None = None
    ttl_seconds:      int | None = None

class BaseAgent(ABC):
    name:             str       = ""
    authority_level:  AuthorityLevel = AuthorityLevel.L1
    allowed_tools:    list[str] = []

    async def run(self, ctx: AgentContext, db=None) -> AgentResult:
        """외부 호출 진입점. pre_check → execute → post_log 순서 보장."""
        await self._pre_check(ctx)
        result = await self.execute(ctx, db)
        await self._post_log(ctx, result)
        return result

    @abstractmethod
    async def execute(self, ctx: AgentContext, db=None) -> AgentResult: ...

    async def check_scope(self, tool_name: str) -> bool:
        if tool_name not in self.allowed_tools:
            from app.core.exceptions import ScopeViolationError
            raise ScopeViolationError(
                f"{self.name}: '{tool_name}' 은 allowed_tools 밖입니다."
            )
        return True
```

```
    async def load_global_context(self) -> dict:
        # Phase 3: 실제 Context 로드 로직 구현
        return {}

    async def trigger_escalation(
        self, trigger_type: str, risk_level: RiskLevel,
        ctx: AgentContext, detail: str = ""
    ) -> None:
        # Phase 3: Escalation 발송 로직 구현
        pass

    async def _pre_check(self, ctx: AgentContext) -> None:
        await self.load_global_context()
        if self.authority_level == AuthorityLevel.L1 and not ctx.payload.get("approved"):
            from app.core.exceptions import ApprovalRequiredError
            raise ApprovalRequiredError(
                f"{self.name}: L1 Agent — Human 승인 필요 (trace_id={ctx.trace_id})"
            )

    async def _post_log(self, ctx: AgentContext, result: AgentResult) -> None:
        # Phase 3: AuditLogger.log() 호출 구현
        pass
```

사용 규칙

```
# ✅ 올바른 사용
class MyAgent(BaseAgent):
    name                = "MyAgent"
    authority_level = AuthorityLevel.L1
    allowed_tools     = ["llm_call", "db_query"]

    async def execute(self, ctx: AgentContext, db=None) -> AgentResult:
        await self.check_scope("llm_call")
        return AgentResult(success=True, data={}, trace_id=ctx.trace_id)

# ❌ 절대 금지
class BadAgent:                          # BaseAgent 상속 없음 — 금지
    def _execute(self): ...          # _execute() 직접 사용 금지
```

② llm_provider.py

경로: backend/src/core/llm_provider.py

```
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import AsyncIterator

@dataclass
class LLMResponse:
    content:            str
    model:              str
    prompt_tokens:      int
```

```
    completion_tokens: int
    cost_usd:          float

class LLMProvider(ABC):
    @abstractmethod
    async def complete(self, messages: list[dict], model: str,
                       max_tokens: int = 1000, temperature: float = 0.7) -> LLMResponse: ...

class ClaudeProvider(LLMProvider):
    def __init__(self, api_key: str):
        import anthropic
        self._client = anthropic.AsyncAnthropic(api_key=api_key)

    async def complete(self, messages, model="claude-sonnet-4-6",
                       max_tokens=1000, temperature=0.7) -> LLMResponse:
        resp = await self._client.messages.create(
            model=model, max_tokens=max_tokens,
            temperature=temperature, messages=messages
        )
        return LLMResponse(
            content=resp.content[0].text, model=model,
            prompt_tokens=resp.usage.input_tokens,
            completion_tokens=resp.usage.output_tokens,
            cost_usd=0.0    # Phase 3: 모델별 단가 계산 구현
        )

class LLMService:
    """Agent 코드에서 직접 사용하는 단일 진입점."""
    FORBIDDEN_PATTERNS = ["ignore previous", "disregard all", "system override"]

    def __init__(self, provider: LLMProvider):
        self._provider = provider

    async def complete(self, messages: list[dict], model: str,
                       max_tokens: int = 1000, temperature: float = 0.7,
                       agent_code: str = "", tenant_id: str = "") -> LLMResponse:
        self._sanitize(messages)
        response = await self._provider.complete(messages, model, max_tokens, temperature)
        await self._log_cost(response, agent_code, tenant_id)
        return response

    def _sanitize(self, messages: list[dict]) -> None:
        for msg in messages:
            content = str(msg.get("content", "")).lower()
            for pattern in self.FORBIDDEN_PATTERNS:
                if pattern in content:
                    raise PromptInjectionError(f"Injection 패턴 감지: ´{pattern}´")

    async def _log_cost(self, resp: LLMResponse, agent_code: str, tenant_id: str):
        # Phase 3: llm_call_logs INSERT 구현
        pass
```

```
class PromptInjectionError(Exception): ...
```

사용 규칙

```
# ✅ 올바른 사용 — LLMService 경유
response = await llm_service.complete(
    messages=[{"role": "user", "content": "..."}],
    model="claude-sonnet-4-6",
    agent_code=self.name,
)

# ❌ 절대 금지 — SDK 직접 호출
import anthropic
anthropic.Anthropic().messages.create(...)    # 금지
```

---

③ orchestrator_agent.py

경로: backend/src/agents/orchestrator_agent.py

```
from src.core.base_agent import BaseAgent, AgentContext, AgentResult, AuthorityLevel

class OrchestratorAgent(BaseAgent):
    name                = "Orch"
    authority_level = AuthorityLevel.L1
    allowed_tools     = ["api_call_internal", "send_notification"]
    _registry: dict[str, BaseAgent] = {}

    @classmethod
    def register(cls, pattern: str, agent: BaseAgent) -> None:
        """신규 Agent 완성 후 반드시 호출."""
        cls._registry[pattern] = agent

    async def execute(self, ctx: AgentContext, db=None) -> AgentResult:
        await self.load_global_context()
        target = self._route(ctx)
        if not target:
            return AgentResult(
                success=False, data={}, trace_id=ctx.trace_id,
                message=f"라우팅 실패: {ctx.payload.get('action', '')}"
            )
        if not await self._check_gate(ctx):
            return AgentResult(
                success=False, data={}, trace_id=ctx.trace_id,
                requires_approval=True, message="Human 승인 대기 중 (P1 Pre-approval)"
            )
        from uuid import uuid4
        sub_ctx = AgentContext(
            trace_id=ctx.trace_id, agent_code=target.name,
            user_id=ctx.user_id, context_version=ctx.context_version,
            risk_level=ctx.risk_level, payload=ctx.payload,
            parent_trace_id=ctx.trace_id,
        )
        return await target.run(sub_ctx, db)
```

```
    def _route(self, ctx: AgentContext) -> BaseAgent | None:
        action = ctx.payload.get("action", "")
        for pattern, agent in self._registry.items():
            if action.startswith(pattern.rstrip("*")):
                return agent
        return None

    async def _check_gate(self, ctx: AgentContext) -> bool:
        # Phase 3: approval_requests 테이블 조회 구현
        return ctx.payload.get("approved", False)
```

사용 규칙

```
# ✅ 등록 — main.py 또는 앱 시작 시 1회 실행
OrchestratorAgent.register("mymodule.*", MyModuleAgent())

# ✅ 호출 — 반드시 Orch 경유
result = await orchestrator.run(ctx, db)

# ❌ 절대 금지
await MyModuleAgent().run(ctx, db)     # Orch 경유 없이 직접 호출 금지
```

---

④ governance_models.py – Governance DB 스키마

경로: backend/src/db/governance_models.py

HITL 패턴(P1/P2/P3)에 필요한 3개 테이블. 기존 12개 테이블과 중복 생성 금지.

```
from enum import Enum
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, JSON, Boolean, DateTime, func
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from app.models.base import TenantBaseModel

class ApprovalStatus(str, Enum):
    PENDING = "PENDING"; APPROVED = "APPROVED"
    REJECTED = "REJECTED"; ESCALATED = "ESCALATED"; REVISED = "REVISED"

class ApprovalRequest(TenantBaseModel):
    """P1 Pre-approval — 실행 전 Human 승인 요청."""
    __tablename__ = "approval_requests"
    agent_code:        Mapped[str]        = mapped_column(String(50),   nullable=False)
    action_type:       Mapped[str]        = mapped_column(String(100),  nullable=False)
    payload:           Mapped[dict]       = mapped_column(JSON,         default=dict)
    status:            Mapped[str]        = mapped_column(String(20),   default=ApprovalStatus.PENDING)
    approver:          Mapped[str|None]   = mapped_column(String(100),  nullable=True)
    rejection_reason:  Mapped[str|None]   = mapped_column(Text,         nullable=True)
    trace_id:          Mapped[str]        = mapped_column(String(36),   nullable=False, index=True)

class ReviewRequest(TenantBaseModel):
    """P3 Post-review — 산출물 완료 후 Human 검토 요청."""
    __tablename__ = "review_requests"
```

```
    agent_code:       Mapped[str]         = mapped_column(String(50),   nullable=False)
    artifact_type:  Mapped[str]         = mapped_column(String(100),  nullable=False)
    artifact_id:     Mapped[str]         = mapped_column(String(36),   nullable=False)
    status:            Mapped[str]         = mapped_column(String(20),   default="PENDING")
    reviewer:         Mapped[str|None]  = mapped_column(String(100),  nullable=True)
    revision_note:  Mapped[str|None]  = mapped_column(Text,            nullable=True)
    trace_id:         Mapped[str]         = mapped_column(String(36),   nullable=False,  index=True)

class  OverrideLog(TenantBaseModel):
    """P2  On-risk — 실행 중 강제 중단 이벤트. 수정·삭제 금지."""
    __tablename__  =  "override_logs"
    triggered_by:            Mapped[str]   =  mapped_column(String(100),  nullable=False)
    trigger_reason:          Mapped[str]   =  mapped_column(Text,            nullable=False)
    affected_agents:        Mapped[list]  =  mapped_column(JSON,            default=list)
    action_taken:            Mapped[str]   =  mapped_column(Text,            nullable=False)
    governance_notified:  Mapped[bool]  =  mapped_column(Boolean,        default=False)
    trace_id:                  Mapped[str]   =  mapped_column(String(36),   nullable=False,  index=True)
```

⑤ governance.ts – 공통 타입 정의

경로: frontend/src/types/governance.ts

```
export  type  AuthorityLevel  =  "L0"  |  "L1"  |  "L2"  |  "L3"
export  type  RiskLevel          =  "LOW"  |  "MEDIUM"  |  "HIGH"  |  "CRITICAL"
export  type  MessageType       =  "REQUEST"  |  "RESPONSE"  |  "ESCALATION"  |  "NOTIFICATION"
export  type  ApprovalStatus  =  "PENDING"  |  "APPROVED"  |  "REJECTED"  |  "ESCALATED"  |  "REVISED"

export  interface  AgentMessage  {
   message_id:              string;   sender_agent:       string
   receiver_agent:       string;   message_type:       MessageType
   payload:                  Record<string,  unknown>
   context_version:     string;   risk_level:          RiskLevel
   requires_approval:  boolean;  created_at:          string
   trace_id:                 string
   parent_message_id?:  string;  ttl_seconds?:       number
}

export  interface  AgentRequest  {
   agent_code:           string;     action:       string
   payload:                Record<string,  unknown>
   risk_level:           RiskLevel;  trace_id?:  string
   context_version?:  string
}

export  interface  AgentResponse  {
   success:                  boolean;   data:          unknown
   trace_id:                string;     risk_level:  RiskLevel
   requires_approval:  boolean;   message?:     string
   error?:  {  code:  string;  message:  string  }
}

export  interface  ApprovalState  {
```

```
  traceId: string; agentCode: string; actionType: string
  requestedAt: string; status: ApprovalStatus
  resolvedAt?: string; reason?: string
}
```

사용 규칙

```
// ✅ governance.ts 에서 import
import type { AgentRequest, RiskLevel } from "../types/governance"

// ❌ 다른 파일에서 재선언 금지
interface AgentRequest { ... }    // 금지
```

---

⑥ useAgent.ts

경로: frontend/src/hooks/useAgent.ts

```
import { useState, useCallback } from "react"
import { v4 as uuidv4 } from "uuid"
import type { AgentRequest, AgentResponse, ApprovalState } from "../types/governance"

export function useAgent() {
  const [isLoading,       setIsLoading]       = useState(false)
  const [error,           setError]           = useState<string | null>(null)
  const [approvalState, setApprovalState] = useState<ApprovalState | null>(null)

  const run = useCallback(async (req: AgentRequest): Promise<AgentResponse> => {
    setIsLoading(true); setError(null)
    const traceId = req.trace_id ?? uuidv4()
    try {
      const res = await fetch("/api/v1/agents/invoke", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ ...req, trace_id: traceId }),
      })
      const data: AgentResponse = await res.json()
      if (data.requires_approval) {
        setApprovalState({
          traceId, agentCode: req.agent_code, actionType: req.action,
          requestedAt: new Date().toISOString(), status: "PENDING",
        })
      }
      return data
    } catch (err) {
      const msg = err instanceof Error ? err.message : "Agent 호출 오류"
      setError(msg); throw err
    } finally {
      setIsLoading(false)
    }
  }, [])

  return { run, isLoading, error, approvalState, clearApproval: () => setApprovalState(null) }
}
```

사용 규칙

```
// ✅ 올바른 사용
const { run, isLoading } = useAgent()
await run({ agent_code: "Back-A", action: "mymodule.create", payload: {}, risk_level: "LOW" })

// ❌ 절대 금지
fetch("/api/v1/agents/invoke", { ... })      // 직접 호출 금지
```

---

⑦ ApprovalGate.tsx

경로: frontend/src/components/ApprovalGate.tsx

패턴	타이밍	UI 행동
P1	실행 전	승인 버튼 클릭 전까지 실행 블로킹
P2	실행 중	전체 화면 경고 모달 + 즉시 중단
P3	완료 후	결과 검토 패널 + 승인/반려/수정 요청

```
import { useState } from "react"
import type { ApprovalStatus, RiskLevel } from "../types/governance"

interface ApprovalGateProps {
  pattern:       "P1" | "P2" | "P3";   level:          "L1" | "L2" | "L3"
  traceId:       string;                      agentCode:     string
  actionLabel:  string;                      riskLevel?:   RiskLevel
  onApprove:     (traceId: string) => void | Promise<void>
  onReject?:     (traceId: string, reason: string) => void | Promise<void>
  onRevise?:     (traceId: string, note: string)     => void | Promise<void>
  children?:     React.ReactNode
}

export function ApprovalGate({ pattern, level, traceId, agentCode,
  actionLabel, riskLevel = "LOW", onApprove, onReject, onRevise, children,
}: ApprovalGateProps) {
  const [status, setStatus]                  = useState<ApprovalStatus>("PENDING")
  const [isProcessing, setIsProcessing] = useState(false)
  const [reason, setReason]                  = useState("")

  const handleApprove = async () => {
    setIsProcessing(true); await onApprove(traceId)
    setStatus("APPROVED"); setIsProcessing(false)
  }
  const handleReject = async () => {
    if (!reason.trim()) return
    setIsProcessing(true); await onReject?.(traceId, reason)
    setStatus("REJECTED"); setIsProcessing(false)
  }

  if (pattern === "P1" && status === "PENDING") return (
```

```
    <div className="border-2 border-yellow-400 rounded-lg p-4 bg-yellow-50">
      <p className="font-bold text-yellow-600 mb-2">◆ 사전 승인 필요 [{level} · {riskLevel}]</p>
      <p className="text-sm mb-3"><strong>{agentCode}</strong>가 <strong>{actionLabel}</strong>을 실행하려
합니다.</p>
      <p className="text-xs text-gray-400 mb-3">trace_id: {traceId}</p>
      <div className="flex gap-2">
        <button onClick={handleApprove} disabled={isProcessing}
          className="px-4 py-2 bg-blue-600 text-white text-sm rounded disabled:opacity-50">
          {isProcessing ? "처리 중..." : "✅ 승인"}
        </button>
        <button onClick={handleReject} disabled={isProcessing || !reason.trim()}
          className="px-4 py-2 bg-red-500 text-white text-sm rounded disabled:opacity-50">
          ✕ 거부
        </button>
      </div>
      <input value={reason} onChange={e => setReason(e.target.value)}
        placeholder="거부 사유 (거부 시 필수)"
        className="mt-2 w-full border px-3 py-1.5 text-sm rounded" />
    </div>
  )

  if (pattern === "P2") return (
    <div className="fixed inset-0 bg-red-900/80 flex items-center justify-center z-50">
      <div className="bg-white rounded-xl p-6 max-w-md w-full mx-4 shadow-2xl">
        <p className="text-red-600 text-xl font-bold mb-2">🚨 긴급 중단</p>
        <p className="text-sm mb-4"><strong>{agentCode}</strong> 실행 중 Risk 감지 — 자동 중단.</p>
        <p className="text-xs text-gray-400 mb-4">trace_id: {traceId}</p>
        <div className="flex gap-2">
          <button onClick={handleApprove}
            className="px-4 py-2 bg-orange-500 text-white text-sm rounded">▶ 재개</button>
          <button onClick={handleReject}
            className="px-4 py-2 bg-gray-700 text-white text-sm rounded">□ 롤백</button>
        </div>
      </div>
    </div>
  )

  if (pattern === "P3" && status === "PENDING") return (
    <div className="border border-gray-200 rounded-lg p-4 bg-gray-50">
      <p className="font-bold text-gray-600 mb-3">📋 검토 요청 [{agentCode}]</p>
      {children}
      <div className="flex gap-2 mt-4">
        <button onClick={handleApprove}
          className="px-3 py-1.5 bg-green-600 text-white text-sm rounded">✅ 확정</button>
        <button onClick={() => { onRevise?.(traceId, reason); setStatus("REVISED") }}
          className="px-3 py-1.5 bg-yellow-500 text-white text-sm rounded">🔄 수정 요청</button>
        <button onClick={handleReject}
          className="px-3 py-1.5 bg-red-500 text-white text-sm rounded">✕ 반려</button>
      </div>
      <textarea value={reason} onChange={e => setReason(e.target.value)}
        placeholder="수정 요청 또는 반려 사유" rows={2}
        className="mt-2 w-full border px-3 py-1.5 text-sm rounded resize-none" />
    </div>
```

```
  )

  if (status === "APPROVED") return <>{children}</>
  return null
}
```

사용 규칙

```
// ✅ 비가역 액션 전 P1 적용
<ApprovalGate pattern="P1" level="L1" traceId={traceId}
  agentCode="Back-A" actionLabel="DB 스키마 변경 (G-06)" riskLevel="HIGH"
  onApprove={handleApprove} onReject={handleReject}>
  <DatabaseChangeForm />
</ApprovalGate>

// ❌ 절대 금지
window.confirm("실행할까요?")    // 승인 처리에 confirm 금지
```

---

5. Phase 4 — Frontend 공통 기반

5.1 Frontend 디렉토리 구조

```
frontend/
├── tsconfig.json              TypeScript + path alias 설정
├── .eslintrc.json             ESLint 규칙
├── .prettierrc                Prettier 포맷 규칙
├── next.config.ts             Next.js 설정
└── src/
    ├── types/
    │   ├── api.ts             API 공통 응답 타입 (Backend schemas/base.py 와 동일 구조)
    │   ├── auth.ts            인증 관련 타입
    │   └── governance.ts      ← Phase 3 에서 구현 완료
    ├── constants/
    │   └── api.ts             API_BASE_URL · 엔드포인트 상수
    ├── lib/
    │   ├── axios.ts           axios 인스턴스 + 토큰 인터셉터
    │   └── queryClient.ts     React Query 설정
    ├── hooks/
    │   ├── useAuth.ts
    │   ├── usePagination.ts
    │   └── useAgent.ts        ← Phase 3 에서 구현 완료
    ├── components/
    │   ├── common/
    │   │   ├── ErrorBoundary.tsx
    │   │   └── LoadingSpinner.tsx
    │   └── ApprovalGate.tsx   ← Phase 3 에서 구현 완료
    └── app/
        ├── providers.tsx      전역 Provider
        └── layout.tsx         루트 레이아웃
```

5.2 주요 파일

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "strict": true,
    "paths": {
      "@/*":                  ["./src/*"],
      "@/components/*": ["./src/components/*"],
      "@/types/*":        ["./src/types/*"],
      "@/hooks/*":        ["./src/hooks/*"]
    }
  }
}
```

types/api.ts — Backend schemas/base.py와 동일 구조 유지

```
export interface ApiResponse<T> {
  success: boolean;
  data: T | null;
  message: string;
}

export interface PaginatedResponse<T> {
  items: T[];
  total: number;
  page: number;
  size: number;
  total_pages: number;
}
```

lib/axios.ts

```
import axios from "axios"

const apiClient = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  timeout: 10000,
  headers: { "Content-Type": "application/json" },
})

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem("access_token")
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
```

```
        localStorage.removeItem("access_token")
        window.location.href = "/login"
      }
      return Promise.reject(error)
    }
)

export default apiClient
```

lib/queryClient.ts

```
import { QueryClient } from "@tanstack/react-query"

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 1000 * 60 * 5, gcTime: 1000 * 60 * 10,
               retry: 1, refetchOnWindowFocus: false },
    mutations: { retry: 0 },
  },
})
```

app/providers.tsx

```
"use client"
import { QueryClientProvider } from "@tanstack/react-query"
import { queryClient } from "@/lib/queryClient"

export function Providers({ children }: { children: React.ReactNode }) {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  )
}
```

6. Phase 5 — 통합 연결 및 검증

6.1 인프라 실행

```
cp .env.example .env            # 환경변수 실제 값 입력
make up                          # 컨테이너 실행
docker compose ps                # 상태 확인
```

6.2 Backend 검증

```
make migrate                     # DB 마이그레이션
docker compose logs -f backend

# API 문서 확인
# → http://localhost:8000/docs    (Swagger)
# → http://localhost:8000/redoc   (ReDoc)

curl http://localhost:8000/health    # {"status": "ok"}
```

6.3 Frontend 검증

```
# → http://localhost:3000

docker compose exec frontend npx tsc --noEmit     # TypeScript 타입 체크
docker compose exec frontend npm run lint          # ESLint 검사
```

6.4 Backend 정적 분석

```
docker compose exec backend bash
ruff check app/
mypy app/
pytest app/tests/
```

7. 전체 작업 체크리스트

Phase 1 — 인프라

#	작업 항목	완료
1	docker-compose.yml 작성	☐
2	.env.example 작성 (LLM API Key 포함)	☐
3	.gitignore 작성	☐
4	Makefile 작성 (up/down/migrate/lint 등)	☐

Phase 2 — Backend 공통 기반

#	작업 항목	완료
5	core/config.py — Pydantic BaseSettings	☐
6	core/logging.py — 공통 로거	☐
7	db/base.py — DeclarativeBase	☐
8	db/session.py — async engine + get_db	☐
9	Alembic 초기화	☐
10	models/base.py — **UUID PK** TenantBaseModel + SoftDeleteMixin	☐
11	schemas/base.py — ApiResponse / PaginatedResponse	☐
12	core/cache.py — Redis 래퍼	☐
13	core/exceptions.py — 커스텀 예외 + ScopeViolationError	☐
14	core/security.py — JWT + require_role + api_success/api_error	☐
15	core/audit.py — create_audit_log	☐
16	api/deps.py — get_database + PaginationParams	☐
17	main.py — FastAPI 앱 진입점	☐

Phase 3 — AI Agent Governance 기반

#	작업 항목	완료
18	core/base_agent.py — BaseAgent + AgentMessage 12 개 필드	☐
19	core/llm_provider.py — LLMService + Injection 방어	☐
20	agents/orchestrator_agent.py — register() + 라우팅	☐
21	db/governance_models.py — approval / review / override 테이블	☐
22	types/governance.ts — 단일 타입 출처	☐
23	hooks/useAgent.ts — trace_id 자동 생성	☐

#	작업 항목	완료
24	components/ApprovalGate.tsx — P1/P2/P3 패턴	□

Phase 4 — Frontend 공통 기반

#	작업 항목	완료
25	tsconfig.json — path alias (@/)	□
26	.eslintrc.json / .prettierrc	□
27	types/api.ts — Backend 응답 구조와 동일	□
28	lib/axios.ts — 인스턴스 + 토큰 인터셉터	□
29	lib/queryClient.ts — React Query 설정	□
30	hooks/useAuth.ts / hooks/usePagination.ts	□
31	components/common/ — ErrorBoundary / LoadingSpinner	□
32	app/providers.tsx / app/layout.tsx	□

Phase 5 — 통합 검증

#	검증 항목	방법	완료
33	모든 컨테이너 정상 실행	docker compose ps	□
34	DB 마이그레이션 성공	make migrate	□
35	Backend API 문서 접속	:8000/docs	□
36	Frontend 앱 접속	:3000	□
37	TypeScript 타입 오류 없음	tsc --noEmit	□
38	ESLint 오류 없음	npm run lint	□
39	Python 타입 체크 통과	mypy app/	□
40	BaseAgent 상속 테스트 통과	pytest	□
41	OrchestratorAgent.register() 라우팅 테스트 통과	pytest	□
42	approval_requests 테이블 생성 확인	\dt	□
43	API CORS 연결 확인 (Frontend → Backend)	브라우저 Network	□

> ✅ 43 개 항목 전체 완료 → Phase 3 WBS 3.1 System Prompt 설계 착수 가능

8. Boilerplate 수정 & 운영 절차

8.1 수정 전 확인 사항

```
1. 어떤 파일을 수정하는가?
2. 그 파일을 상속·사용하는 하위 파일은 무엇인가?
3. DB 스키마 변경이 포함되는가? → alembic 필요
4. phase3_context.json 업데이트가 필요한가?
```

8.2 파일별 수정 절차

base_agent.py 수정 시

```
# Step 1. 파일 수정 (Cursor 사용)
# Step 2. 영향받는 Agent 파일 확인
grep -r "BaseAgent" backend/src/
# Step 3. Docker 재시작
docker compose -f docker-compose.dev.yml restart backend
# Step 4. 테스트
POST /api/v1/agents/invoke
```

base_model.py 수정 시 (DB 변경 포함)

```
# Step 1. 파일 수정
# Step 2. 영향받는 모델 확인
grep -r "TenantBaseModel" backend/src/
# Step 3. 마이그레이션 생성
docker compose -f docker-compose.dev.yml exec backend \
  alembic revision --autogenerate -m "변경내용 설명"
# Step 4. DB 반영
docker compose -f docker-compose.dev.yml exec backend \
  alembic upgrade head
# Step 5. Docker 재시작
docker compose -f docker-compose.dev.yml restart backend
```

security.py 수정 시

```
# Step 1. 파일 수정
# Step 2. Docker 재시작
# Step 3. 인증 흐름 순서대로 재테스트 (순서 중요!)
① POST /api/v1/auth/login
② POST /api/v1/auth/refresh
③ 각 API Bearer 토큰 테스트
```

llm_provider.py 수정 시

```
# Step 1. 파일 수정
# Step 2. 영향받는 Agent 확인
grep -r "LLMService\|llm_service" backend/src/
# Step 3. Docker 재시작
# Step 4. LLM 호출 테스트
POST /api/v1/agents/invoke
```

database.py 수정 시

```
# Step 1. 파일 수정
```

```
# Step 2. Docker 재시작
# Step 3. 연결 확인
GET /health
```

8.3 자주 쓰는 명령어 모음

```
# 컨테이너 상태 확인
docker compose -f docker-compose.dev.yml ps

# Backend 재시작
docker compose -f docker-compose.dev.yml restart backend

# 마이그레이션 생성
docker compose -f docker-compose.dev.yml exec backend \
  alembic revision --autogenerate -m "설명"

# 마이그레이션 적용
docker compose -f docker-compose.dev.yml exec backend \
  alembic upgrade head

# 마이그레이션 되돌리기
docker compose -f docker-compose.dev.yml exec backend \
  alembic downgrade -1

# DB 테이블 목록 확인
docker compose -f docker-compose.dev.yml exec db \
  psql -U postgres -d myapp -c "\dt"

# Backend 로그 확인
docker compose -f docker-compose.dev.yml logs -f backend
```

8.4 phase3_context.json 업데이트 기준

Boilerplate 수정 후 아래 항목이 바뀌었으면 **반드시** phase3_context.json 도 업데이트한다.

변경 내용	업데이트할 섹션
새 클래스 · 함수 추가	boilerplate[].key_classes 또는 key_functions
do/dont 규칙 변경	boilerplate[].do / boilerplate[].dont
코딩 규칙 변경	code_generation_rules
Agent 템플릿 변경	agent_template
모듈 폴더 구조 변경	module_template
새 API 엔드포인트 추가	api_endpoints.existing

Cursor 에 변경사항 알리는 방법

```
@phase3_context.json

phase3_context.json 이 업데이트 되었어.
```

```
변경된 내용: (변경사항 설명)
앞으로 코드 생성 시 새 규칙을 반영해줘.
```

8.5 수정 완료 후 체크리스트

```
☐ 수정한 파일 저장 확인
☐ 영향받는 하위 파일 확인 및 수정
☐ DB 변경이 있으면 alembic 마이그레이션 완료
☐ docker restart backend 완료
☐ phase3_context.json 업데이트 완료
☐ Cursor 에 변경사항 알림
☐ 관련 API 테스트 완료
☐ 이 문서 업데이트 (필요시)
```

---

9. 참고사항 및 주의점

9.1 의존성 순서를 반드시 지켜야 하는 이유

Python 에서 모듈을 import 할 때 해당 모듈이 의존하는 다른 모듈이 먼저 존재해야 한다. session.py 는 config.py 의 settings 를 import 하므로, config.py 가 먼저 완성되어 있어야 한다. 순서를 무시하면 ImportError 또는 순환 참조(Circular Import)가 발생한다.

9.2 PK 는 반드시 UUID

saas_foundation.json 기준: id UUID PRIMARY KEY DEFAULT gen_random_uuid(). Integer autoincrement 는 절대 사용하지 않는다.

9.3 환경변수 관리

- .env 파일은 절대 Git 에 커밋하지 않는다.
- .env.example 은 실제 값 없이 키만 포함하여 Git 에 커밋한다.
- 새 환경변수 추가 시 .env.example 도 동시에 업데이트한다.
- 코드 내 Secret 하드코딩 절대 금지 — settings.secret_key 경유.

9.4 SQLAlchemy async 사용 시

- asyncpg 드라이버 필수 — DATABASE_URL 에 postgresql+asyncpg:// 형식 사용.
- 모든 DB 쿼리는 async def + await 사용.
- 세션은 반드시 Depends(get_db) 의존성 주입 사용.

9.5 절대 금지 사항 요약

금지 행동	대신 사용
BaseAgent 상속 없이 Agent 생성	BaseAgent 상속 필수
anthropic.Anthropic().messages.create() 직접 호출	LLMService.complete()
Orch 경유 없이 Agent 직접 호출	OrchestratorAgent.run()
Base 직접 상속 DB 모델	TenantBaseModel 상속
Integer PK	UUID PK
db.delete(record) 물리 삭제	.soft_delete()
fetch() 직접 Agent 호출	useAgent().run()
governance.ts 타입 재선언	governance.ts 에서 import
window.confirm() 승인 처리	<ApprovalGate> 사용
코드 내 Secret 하드코딩	settings.* 경유

9.6 Cursor AI 참조 지침

```
Phase 3 코드 생성 시 반드시 @참조:

@phase3_context.json        ← 전체 구현 기준
@workflow_context.json      ← Workflow · Gate · Escalation 기준
@governance.json            ← Authority Level 결정 기준
@hitl_context.json          ← P1/P2/P3 패턴 및 SLA 기준
```

문서 연결

궁금한 내용	확인할 문서
"왜 이 기술을 선택했나?"	▤ Framework — 해당 결정 항목
"공통 SaaS 기반 구현 순서는?"	▧ Implementation Guide — Phase 2~4
"네이밍 규칙 · 에러 코드는?"	▨ Quick Reference
"Agent Workflow · Gate 기준은?"	workflow_context.json
"Authority Level 기준은?"	governance.json
"HITL 패턴 SLA 는?"	hitl_context.json
"Phase 3 구현 순서는?"	phase3_context.json